

ELEC6910J

Deep Reinforcement Learning

Lecture 5. Tabular RL I: Value Functions & Bellman Equations

Course Instructor: Ling PAN

Department of Electronic and Computer Engineering

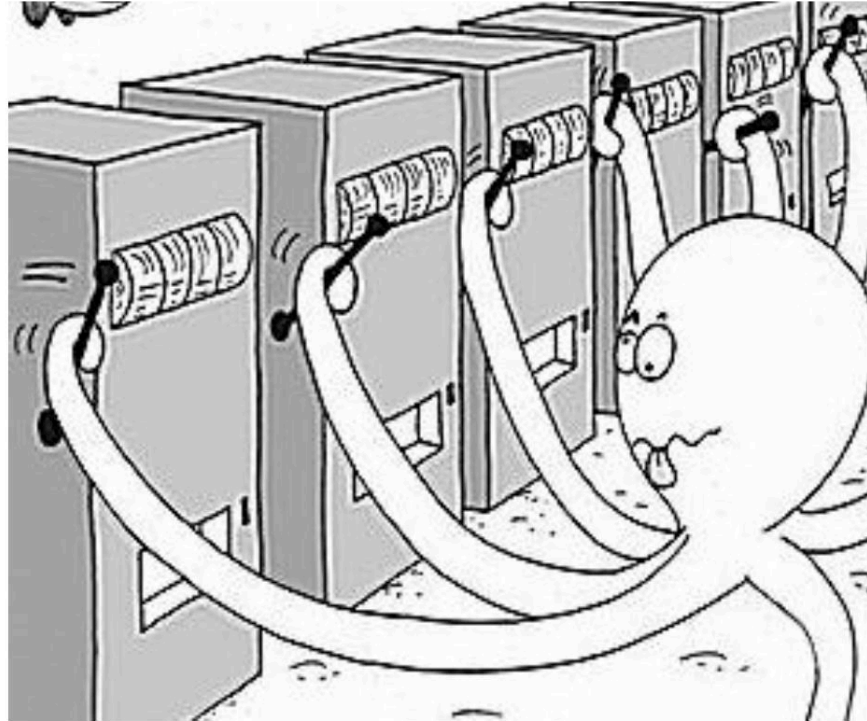
Hong Kong University of Science and Technology

Most slides are adapted from UC Berkeley CS188/CS285, CMU 10-403

Recap: Multi-Armed Bandits



One-armed bandit
(Slot machine)



Multi-armed bandit
(Multiple slot machines)

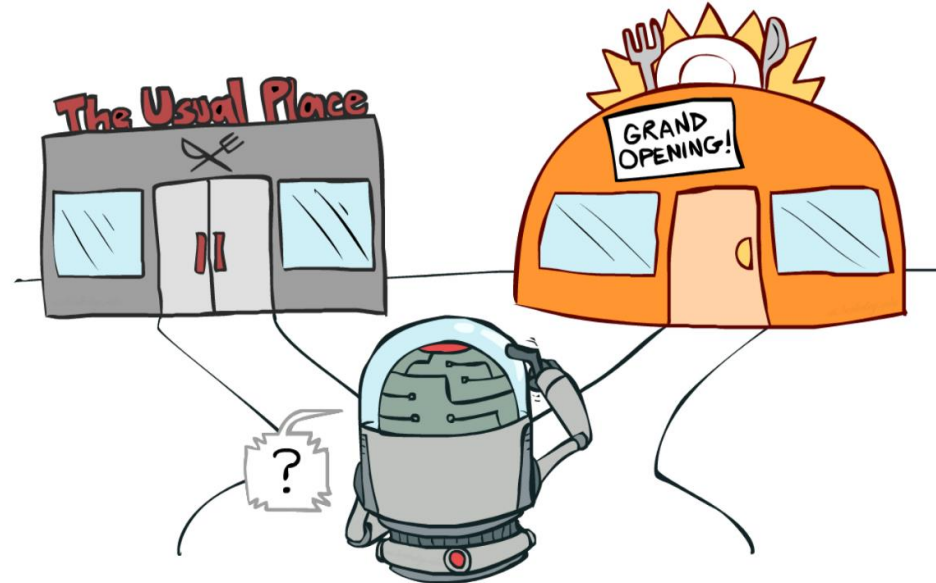
$a_t, r_{t+1}, a_{t+1}, r_{t+2}, \dots,$
The state does not change
(stateless)

Recap: Exploration/Exploitation Dilemma

- Define the greedy at time t as

$$A_t^* = \operatorname{argmax}_a Q_t(a)$$

- If $A_t = A_t^*$, then you are exploiting
- If $A_t \neq A_t^*$, then you are exploring
- You can't do both, but you need to do both
- You can never stop exploring, but maybe you should explore less with time



Recap: Forming Action-Value Estimates

- Stationary bandits (sample average):

- Its action value estimate after $n - 1$ rewards

$$Q_n = \frac{R_1 + R_2 + \dots + R_{n-1}}{n - 1}$$

- Equivalent & efficient calculation

$$Q_{n+1} = Q_n + \frac{1}{n} [R_n - Q_n]$$

- Generalization: This is a standard form for learning/update rules throughout this course:

$$\text{New Estimate} \leftarrow \text{Old Estimate} + \text{Step Size} \times \underbrace{[\text{Target} - \text{Old Estimate}]}_{\text{Error}}$$

- Non-stationary bandits (exponential, recency-weighted average)

$$Q_{k+1} = (1 - \alpha)^k Q_1 + \sum_{i=1}^k \alpha (1 - \alpha)^{k-i} R_i$$

Recap: Epsilon-Greedy Action Selection

- With probability $1 - \epsilon$ select $a_t = \operatorname{argmax}_a Q_t(a)$
- With probability ϵ select a random action (independent of its Q-value estimate)

A simple bandit algorithm

Initialize, for $a = 1$ to k :

$$Q(a) \leftarrow 0$$

$$N(a) \leftarrow 0$$

Repeat forever:

$$A \leftarrow \begin{cases} \operatorname{argmax}_a Q(a) & \text{with probability } 1 - \epsilon \\ \text{a random action} & \text{with probability } \epsilon \end{cases} \quad (\text{breaking ties randomly})$$

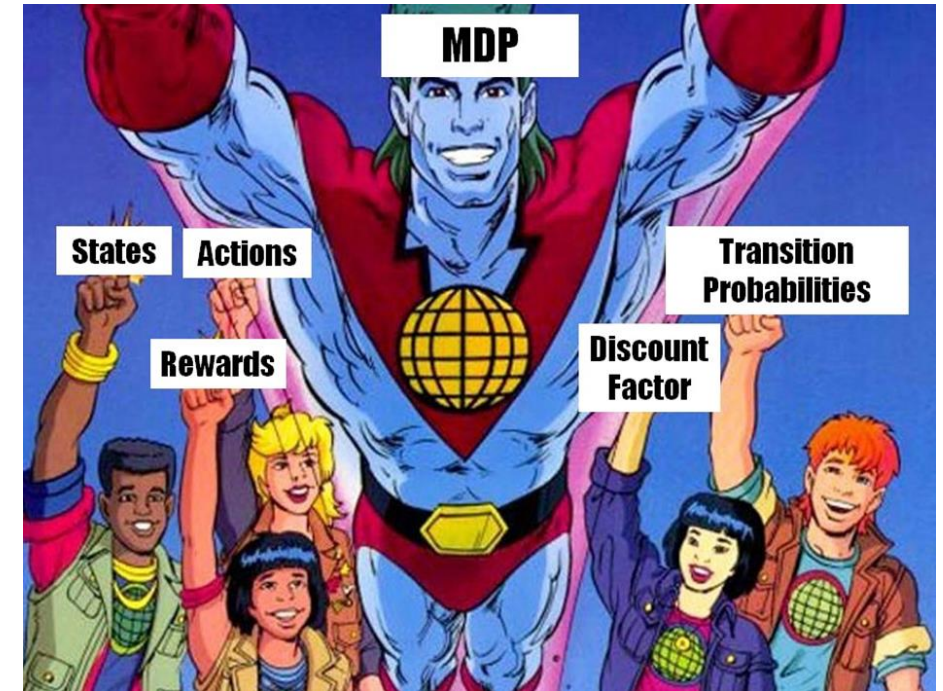
$$R \leftarrow \text{bandit}(A)$$

$$N(A) \leftarrow N(A) + 1$$

$$Q(A) \leftarrow Q(A) + \frac{1}{N(A)} [R - Q(A)]$$

Recap: Markov Decision Process (MDP)

- Definition: (S, A, T, R, γ)
 - A set of states $s \in S$
 - A set of actions $a \in A$
 - A transition function $T(s, a, s')$
 - Probability that a from s leads to s' , i.e., $P(s'|s, a)$
 - Also called the model or the dynamics
 - A reward function $R(s, a, s')$
 - A discount factor γ



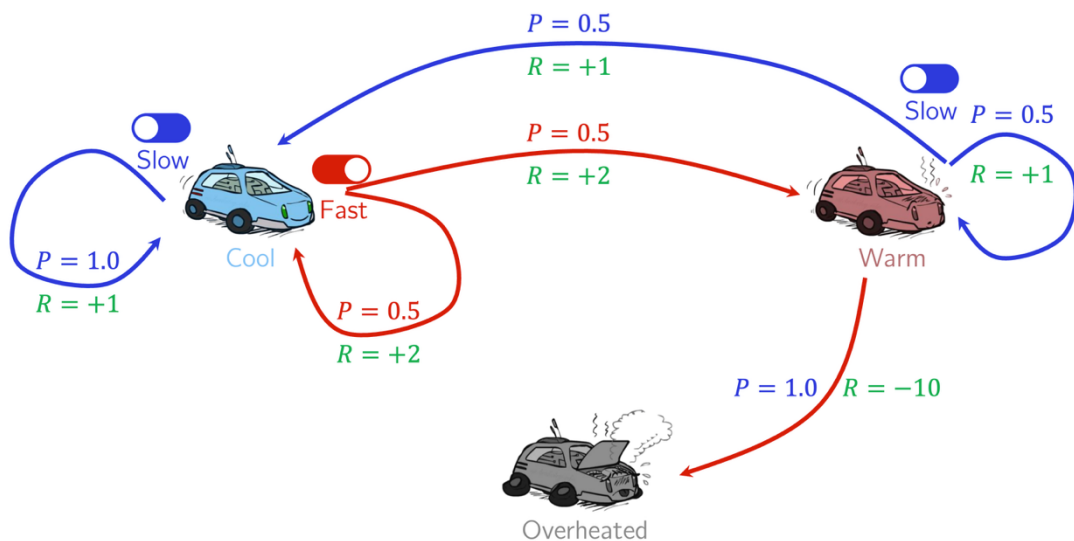
- Property:

$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, A_{t-1} = a_{t-1}, \dots, S_0 = s_0) = P(S_{t+1} = s' | S_t = s_t, A_t = a_t)$$

- Given the present state, the future and the past are independent
→ We should be able to throw away the history once state is known

Recap: Markov Decision Process (MDP)

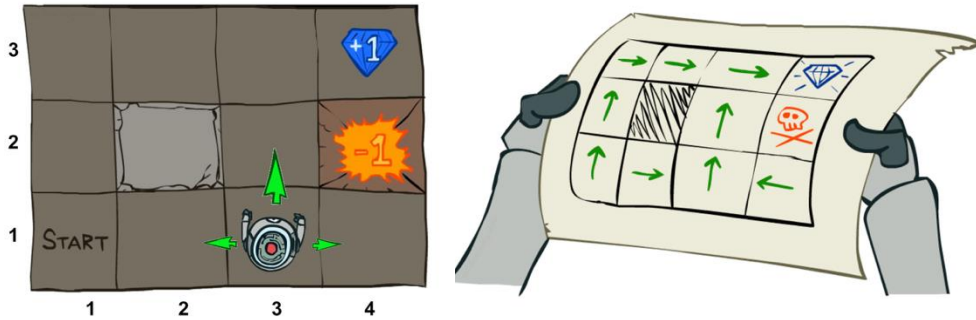
- Example: car racing
 - Transition dynamics
 - Reward function



s	a	s'	$P(s' s, a)$	$R(s, a, s')$
 Cool	 Slow	 Cool	1.0	+1
 Cool	 Fast	 Cool	0.5	+2
 Cool	 Fast	 Warm	0.5	+2
 Warm	 Slow	 Cool	0.5	+1
 Warm	 Slow	 Warm	0.5	+1
 Warm	 Fast	 Overheated	1.0	-10
 Overheated	(end)	 Overheated	1.0	0

Recap: Markov Decision Process (MDP)

- MDP quantities so far:
 - Policy = choice of action for each state



- Utility = sum of discounted rewards

$$G_T = R_{t+1} + \gamma R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$



1

Worth Now



γ

Worth Next Step



γ^2

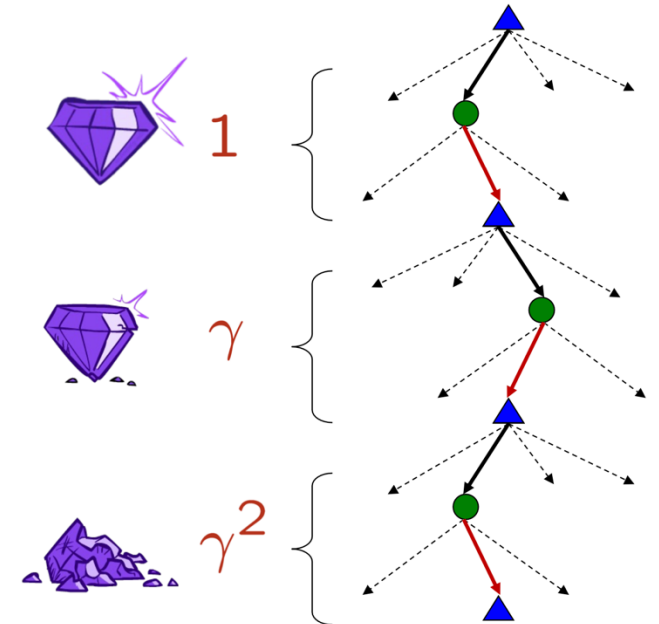
Worth In Two Steps

Recap: Discounted Return

- Definition: the return G_t is the total discounted reward from timestep t

$$G_T = R_{t+1} + \gamma R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- The discount $\gamma \in [0, 1]$ determines the present value of future rewards
 - It is sometimes possible to use undiscounted Markov reward processes (i.e., $\gamma = 1$),
 - It is sometimes possible to use undiscounted Markov reward processes (i.e., $\gamma = 1$), e.g., if all sequences terminate
- The value of receiving reward R after $k + 1$ time-steps is $\gamma^k R$
- This values immediate reward above delayed reward
 - γ close to 0 leads to "myopic" evaluation
 - γ close to 1 leads to "far-sighted" evaluation



Value Functions

- Motivation: we don't know P or R



Which states/actions are good? $\rightarrow$ value function: $V^\pi(s)$ / $Q^\pi(s, a)$ [expected returns]

- State value function for policy π : the expected return starting from state s , and then following policy π

$$\underbrace{V^\pi(s)}_{\text{value of a state}} = \underbrace{\mathbb{E}_\pi}_{\text{expected}} \left[\underbrace{G_t}_{\text{starting at state } s} \mid S_t = s \right] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right]$$

$\rightarrow$ Take the expectation over all possible discounted returns

Note: the value of the terminal state (if any) is always zero

Value Functions

- Motivation: we don't know P or R



Which states/actions are good? $\rightarrow$ value function: $V^\pi(s)$ / $Q^\pi(s, a)$ [expected returns]

- State-action value function for policy π : the expected return starting from state s , taking action a , and then following policy π

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s, A_t = a \right]$$

Value Functions

- Motivation: we don't know P or R



Which states/actions are good? $\rightarrow$ value function: $V^\pi(s)$ / $Q^\pi(s, a)$ [expected returns]

$$\underbrace{V^\pi(s)}_{\text{value of a state}} = \underbrace{\mathbb{E}_\pi}_{\text{expected}} \left[\underbrace{G_t}_{\text{starting at state } s} \mid S_t = s \right] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right]$$

value of a state expected discounted return

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right]$$

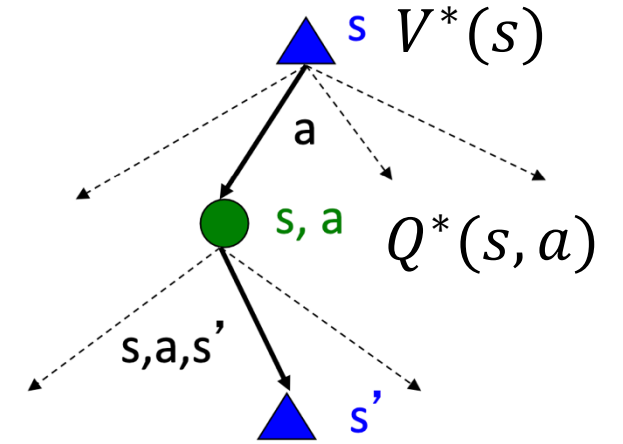
Value Functions

- Optimal value functions:

- State value $V^*(s)$:

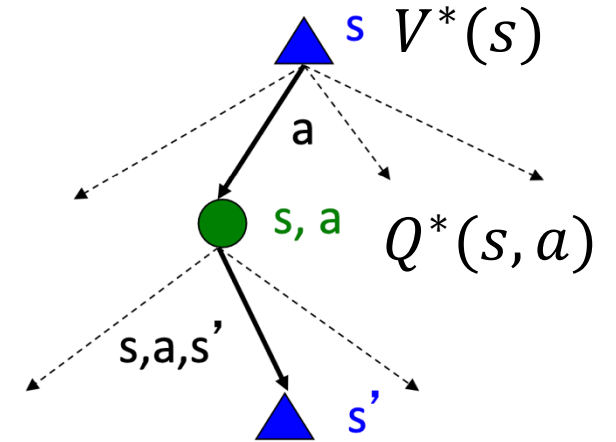
- expected utility starting in s and act optimally

- the maximum value function over all policies $V^*(s) = \max_{\pi} V^{\pi}(s)$



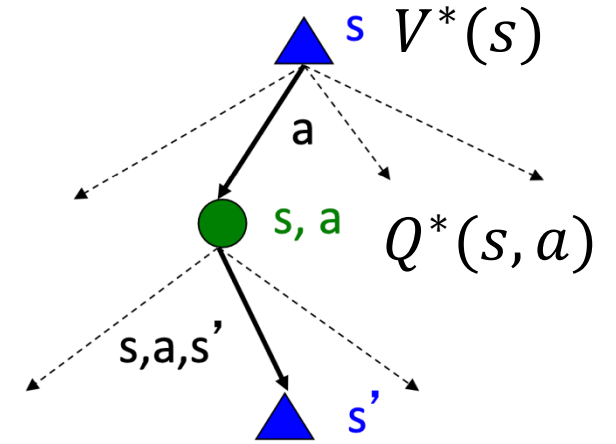
Value Functions

- Optimal value functions:
 - State-action value $Q^*(s, a)$:
 - expected utility starting out having taken action a from state s and (thereafter) acting optimally
 - the maximum action-value function over all policies $Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$



Value Functions

- The optimal policy:
 - $\pi^*(s)$ = optimal action from state s
 - $\pi^*(a|s) > 0$ only where $Q^*(s, a) = \max_b Q^*(s, b)$, i.e., greedy w.r.t. Q^*



Value Functions

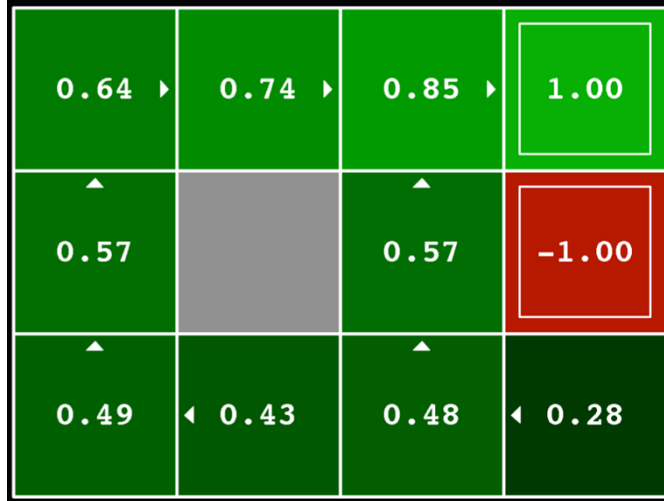
- Solving MDPs:
 - Prediction:
 - Given an MDP (S, A, T, r, γ) and a policy $\pi(a|s) = P[A_t = a|S_t = s]$, find the state and action value functions
 - Optimal control:
 - Given an MDP (S, A, T, r, γ) , find the optimal policy (the planning problem).
 - Compare with the learning problem where rewards and dynamics are unknown.

	state values	action values
prediction	V^π	Q^π
control	V_*	Q_*

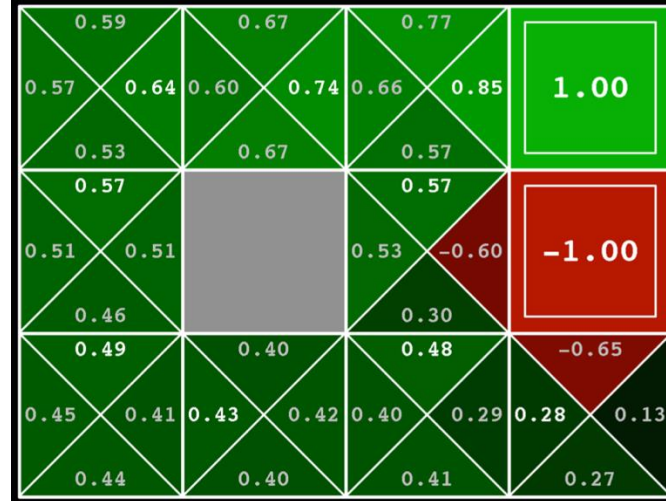
- ✓ State value function for s given a policy π :
$$V^\pi(s) = \mathbb{E}\{G_t | S_t = s, A_{t:\infty} \sim \pi\}$$
- ✓ State-action value function for (s, a) given a policy π :
$$Q^\pi(s, a) = \mathbb{E}\{G_t | S_t = s, A_t = a, A_{t+1:\infty} \sim \pi\}$$
- ✓ Optimal state value function:
$$V^*(s) = \max_{\pi} V^\pi(s)$$
- ✓ Optimal state-action value function:
$$Q^*(s, a) = \max_{\pi} Q^\pi(s, a)$$

Value Functions

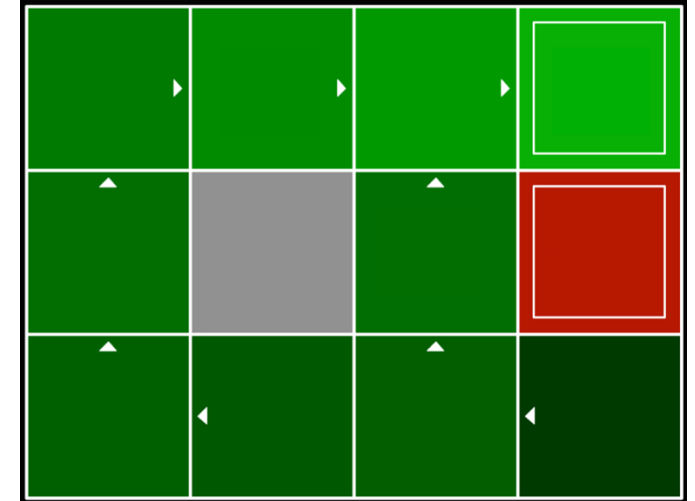
- Example: gridworld



$V^*(s)$



$Q^*(s, a)$



$\pi^*(s)$

(Noise = 0.2, Discount = 0.9, Living reward = 0)

Value Functions

- Recursive relationships for returns

$$\begin{aligned}G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \\&= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \dots) \\&= R_{t+1} + \gamma G_{t+1}\end{aligned}$$



Condition on a state and take expectations

$$\mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s]$$

$$\underbrace{V^\pi(s)}_{\text{value of a state}} = \underbrace{\mathbb{E}_\pi}_{\text{expected}} \left[\underbrace{R_{t+1}}_{\text{immediate reward}} + \underbrace{\gamma V^\pi(S_{t+1})}_{\text{+discounted value of next state}} \right] \underbrace{|S_t = s]}_{\text{if the agent starts at state } s}$$

and uses the policy to choose its actions for all time steps

The value of state s : the average immediate reward that can be obtained, plus the discounted value of the next state that will be visited on average

Value Functions

- Recursive relationships for returns

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \\ &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \dots) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$



Condition on a state and take expectations

$$\mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s]$$

$$\underbrace{V^\pi(s)}_{\text{value of a state}} = \underbrace{\mathbb{E}_\pi}_{\text{expected}} \underbrace{[R_{t+1}]}_{\text{immediate reward}} + \underbrace{\gamma V^\pi(S_{t+1})}_{\text{+discounted value of next state}} \underbrace{|S_t = s]}_{\text{if the agent starts at state } s}$$

and uses the policy to choose its actions for all time steps

? How to compute this?

→ To calculate an expectation, we multiply every possible outcome by its probability of happening, and then sum them all up.

Value Functions

- Recursive relationships for returns

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \\ &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \dots) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$



Condition on a state and take expectations

$$\mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s]$$

$$V^\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma V^\pi(S_{t+1}) | S_t = s]$$

$$= \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma V^\pi(s')]$$



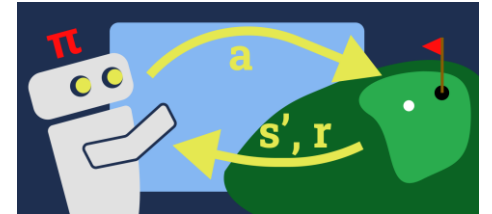
(law of total probability)

Value Functions

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi[R_{t+1} + \gamma V^\pi(S_{t+1}) | S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma V^\pi(s')] \end{aligned}$$

(law of total probability)

- the 1st Layer of randomness: your choice (policy π)
- the 2nd Layer of randomness: the environment's response (dynamics p)
- the actual value of the outcome: If that specific scenario happens, your return is the immediate reward plus the discounted value of wherever you landed next

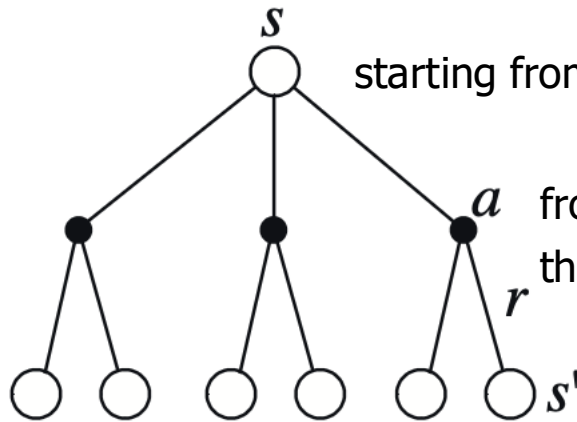


Value Functions

- Bellman equation for V^π :

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma V^\pi(s')]$$

- Relationship between the value of the current state and the values of its successor states
- The value function $V^\pi(s)$ is the unique solution to its Bellman equation



starting from state s , the agent could take any of some set of actions

from each of these actions,
the environment can respond with one of the next states s' and a reward r

Figure. Backup diagram.

look ahead from one state to its possible successor states

- averages over all the possibilities
- weights each by its probability of occurring

Value Functions

- Recursive relationships for returns

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \\ &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \dots) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$



Condition on a state and action and take expectations

$$\mathbb{E}_\pi[G_t | S_t = s, A_t = a] = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s, A_t = a]$$

$Q^\pi(s, a)$

value of a state-action pair (s,a)

expected return

if the agent starts at state s

and chooses action a

and then uses the policy to choose its actions for all time steps

For each state and action,
the Q-function outputs
the expected return
if the agent starts in that state
and takes the action
and then follows the policy forever after

Value Functions

- Recursive relationships for returns

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \\ &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \dots) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$



Condition on a state and action and take expectations

$$\mathbb{E}[G_t | S_t = s, A_t = a] = \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s, A_t = a]$$

$$\begin{aligned} Q^\pi(s, a) &= \mathbb{E}[R_{t+1} + \gamma Q^\pi(S_{t+1}, A_{t+1}) | S_t = s, A_t = a] \\ &= \sum_{s', r} p(s', r | s, a) \left[r + \gamma \sum_{a'} \pi(a' | s') Q^\pi(s', a') \right] \end{aligned}$$

Value Functions

- Bellman equation for Q^π :

$$Q^\pi(s, a) = \sum_{s', r} p(s', r | s, a) \left[r + \gamma \sum_{a'} \pi(a' | s') Q^\pi(s', a') \right]$$

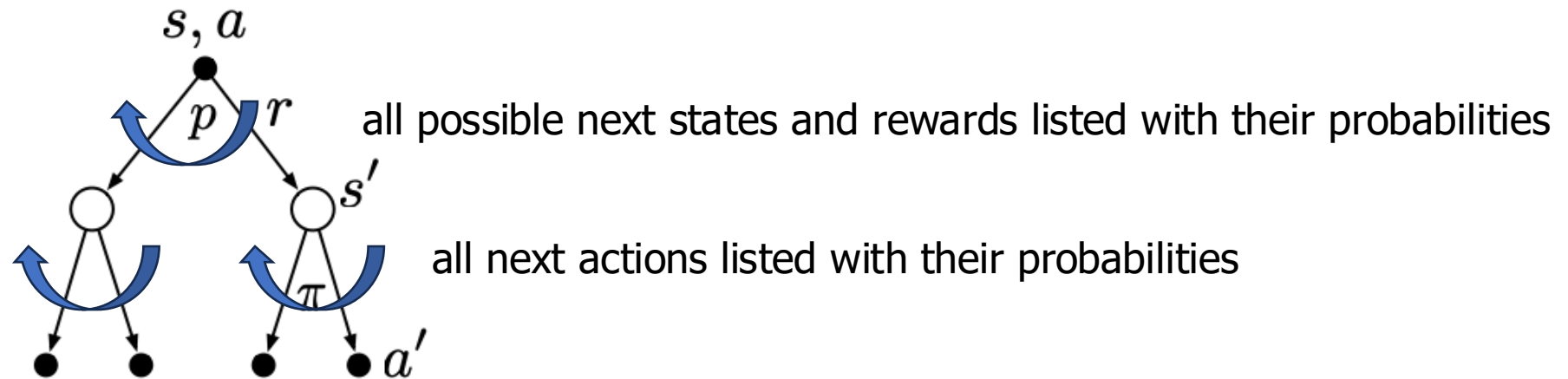
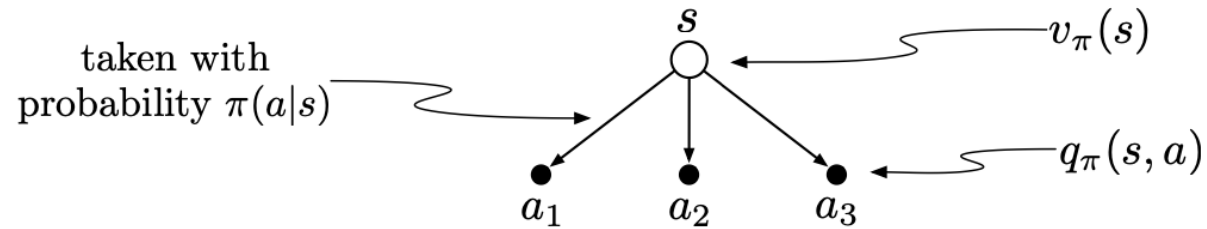


Figure. Backup diagram.

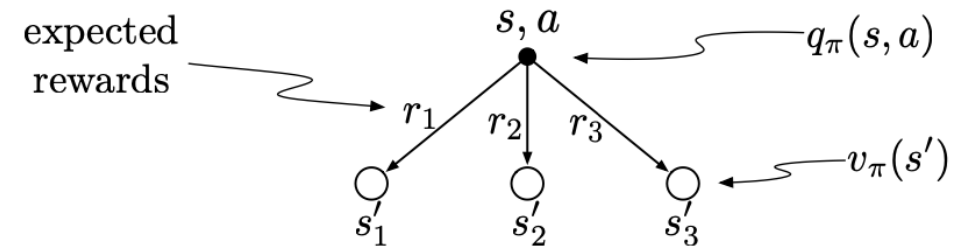
Value Functions

- Relating V^π and Q^π :

$$V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s, a)$$

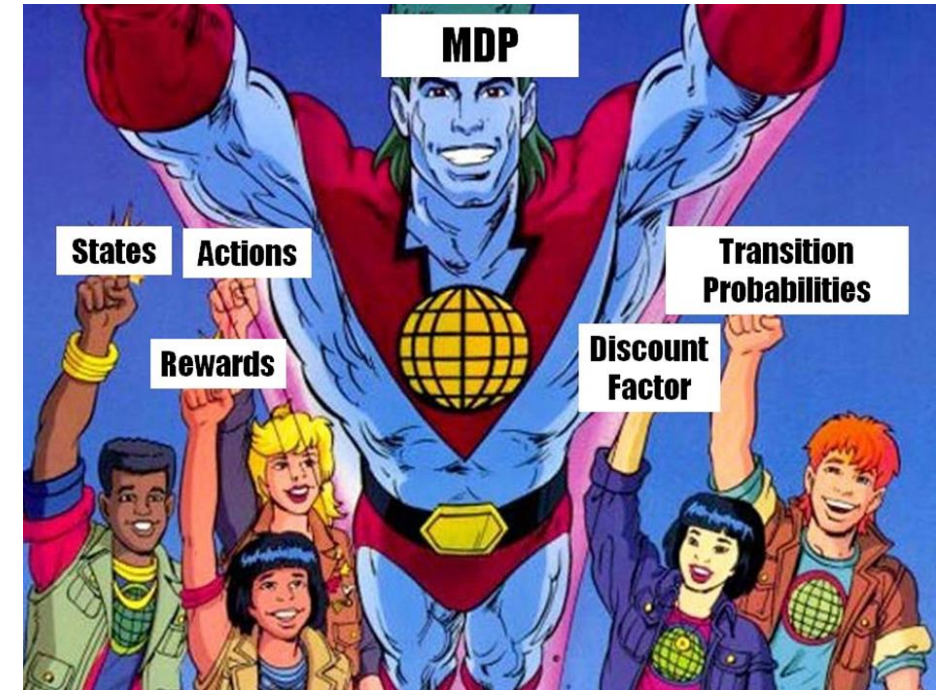


$$Q^\pi(s, a) = \sum_{s', r} p(s', r|s, a) [r + \gamma V^\pi(s')]$$



Markov Decision Process (MDP)

- Definition: (S, A, T, R, γ)
 - A set of states $s \in S$
 - A set of actions $a \in A$
 - A transition function $T(s, a, s')$
 - Probability that a from s leads to s' , i.e., $P(s'|s, a)$
 - Also called the model or the dynamics
 - A reward function $R(s, a, s')$
 - A discount factor γ
- Quantities:
 - Policy: map of states to actions
 - Utility: sum of discounted rewards
 - State values: expected future utility from a state
 - State-action (Q-)values: expected future utility from a state-action pair



Optimal Value Functions

- Optimal state value function: $V^*(s) = \max_{\pi} V^{\pi}(s)$
- Optimal state-action value function: $Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$

Bellman Equation for V

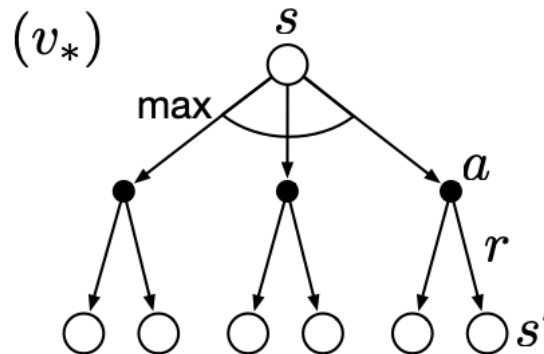
- Bellman expectation equation for V^π :

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V^\pi(s')]$$

- Bellman optimality equation for V^* :

- The value of a state under an optimal policy must equal the expected return for the **best** action from that state

$$V^*(s) = \max_a \left(\sum_{s',r} p(s',r|s,a) [r + \gamma V^*(s')] \right)$$



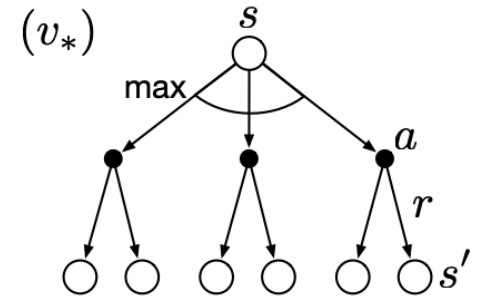
$V^*(s)$ is the unique solution of this system of nonlinear equations

Bellman Equation for V

$$\sum_a \pi(a|s) \dots$$

$$\max_a \dots$$

- Bellman expectation equation for V^π : $V^\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma V^\pi(s')]$
- Bellman optimality equation for V^* : $V^*(s) = \max_a (\sum_{s',r} p(s',r|s,a)[r + \gamma V^*(s')])$



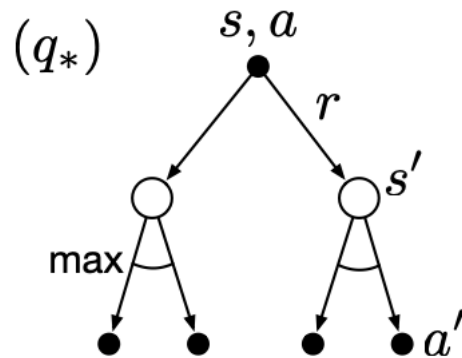
Summary: Bellman equation for Q

- Bellman expectation equation for Q^π :

$$Q^\pi(s, a) = \sum_{s', r} p(s', r | s, a) \left[r + \gamma \sum_{a'} \pi(a' | s') Q^\pi(s', a') \right]$$

- Bellman optimality equation for Q^* :

$$Q^*(s, a) = \sum_{s', r} p(s', r | s, a) \left[r + \gamma \max_{a'} Q^*(s', a') \right]$$



$Q^*(s, a)$ is the unique solution of this system of nonlinear equations

Optimal Value Functions

- Recursive definition of value (Bellman equation)
 - Optimal state value function:

$$V^*(s) = \max_a Q^*(s, a)$$

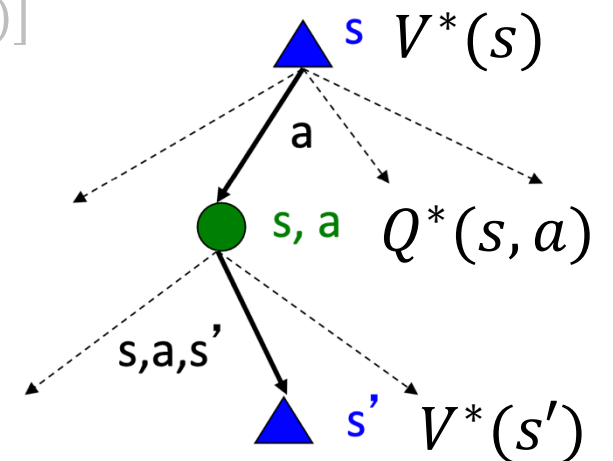
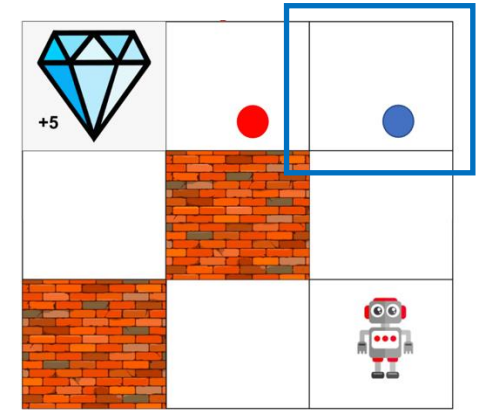
- Example: $V^*(\bullet) = \max_a \{Q^*(\bullet, \uparrow), Q^*(\bullet, \downarrow), Q^*(\bullet, \leftarrow), Q^*(\bullet, \rightarrow)\}$

- Optimal state-action value function:

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

- Therefore:

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$



Optimal Value Functions

- Why value functions are useful?

- If we know $Q^*(s, a)$:

- An optimal policy can be found by directly maximizing over $Q^*(s, a)$:

$$\pi^*(a|s) = \begin{cases} 1, & \text{if } a = \operatorname{argmax}_a Q^*(s, a) \\ 0, & \text{otherwise} \end{cases}$$

- If we know $V^*(s)$:

- An optimal policy can be found from the model dynamics using one step look ahead:

$$\pi^*(a|s) = \begin{cases} 1, & \text{if } a = \operatorname{argmax}_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')] \\ 0, & \text{otherwise} \end{cases}$$

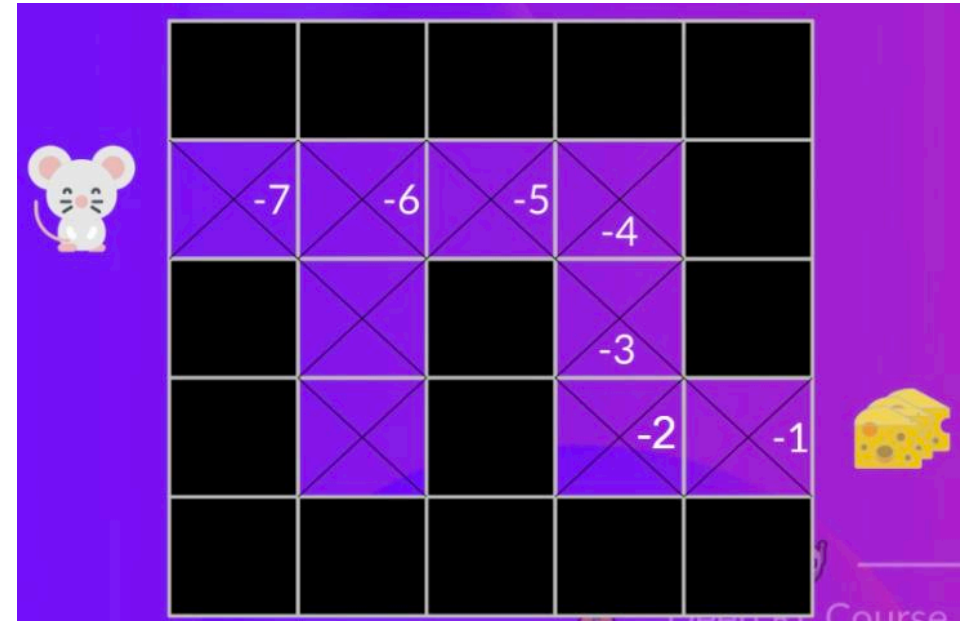
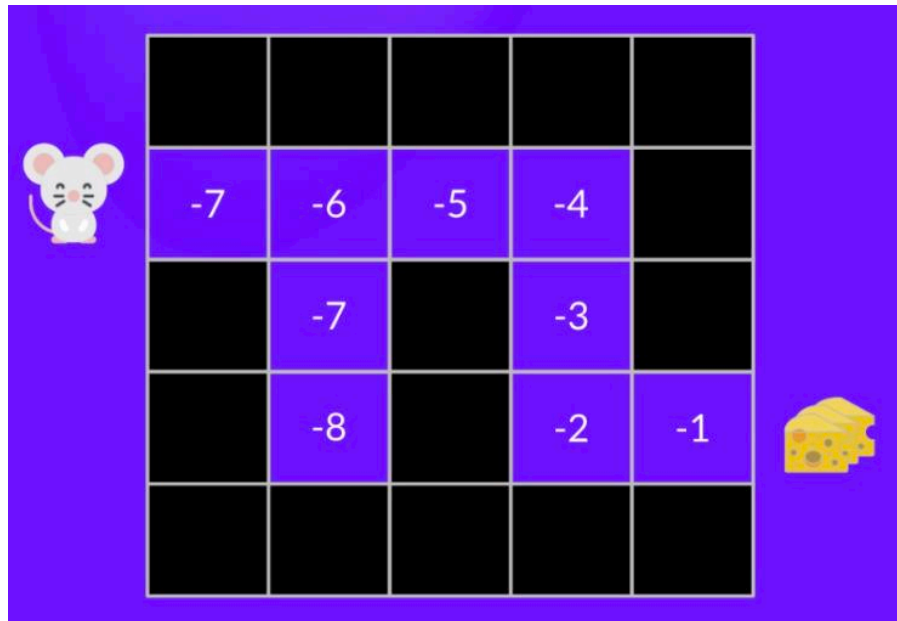
- Link between value and policy:

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$$

- Finding an optimal value function leads to having an optimal policy

Value Functions

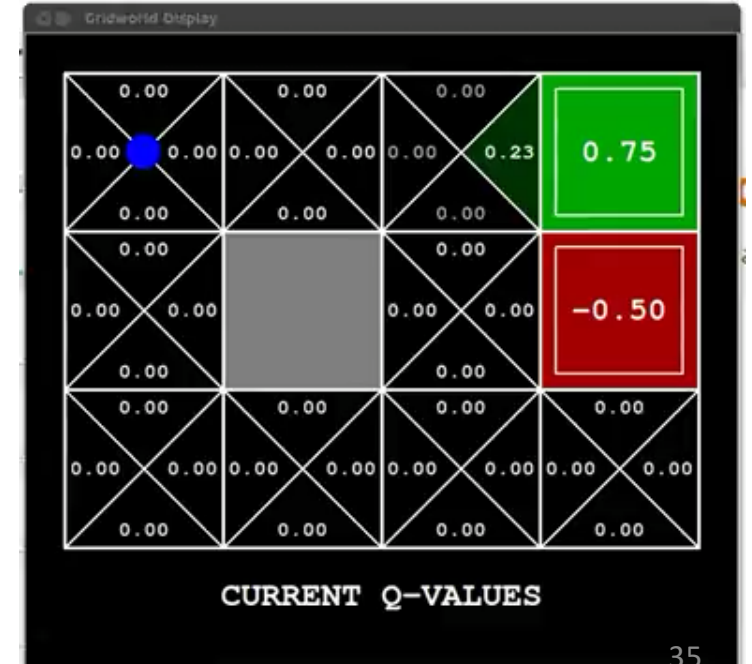
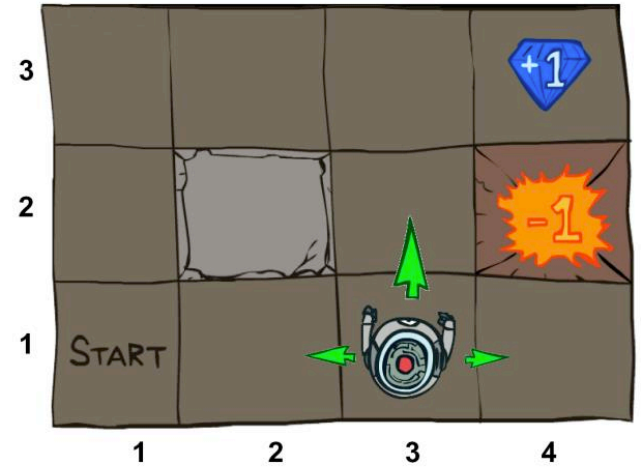
- Two types of value-based methods
 - State value function: calculate the value of a state
 - State-action value function: calculate the value of state-action pair



Problem: How to solve these equations?

- $V^\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V^\pi(s')]$
- $Q^\pi(s,a) = \sum_{s',r} p(s',r|s,a) [r + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s',a')]$
- $V^*(s) = \max_a (\sum_{s',r} p(s',r|s,a) [r + \gamma V^*(s')])$
- $Q^*(s,a) = \sum_{s',r} p(s',r|s,a) [r + \gamma \max_{a'} Q^*(s',a')]$

➤ This lecture: Solving Bellman Equations



ELEC6910J

Deep Reinforcement Learning

Lecture 6. Tabular RL II: Dynamic Programming

Course Instructor: Ling PAN

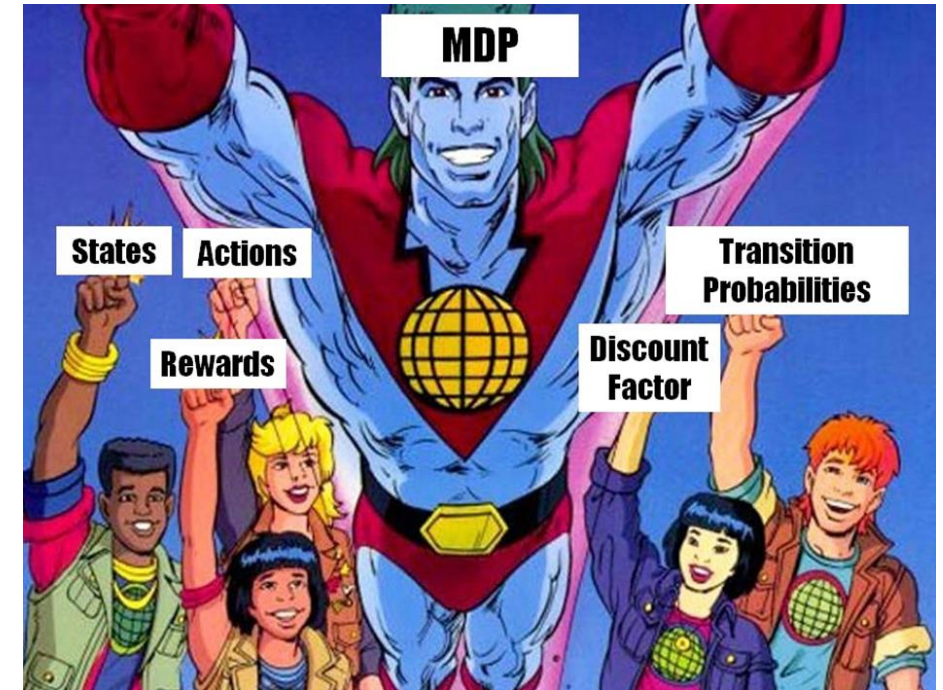
Department of Electronic and Computer Engineering

Hong Kong University of Science and Technology

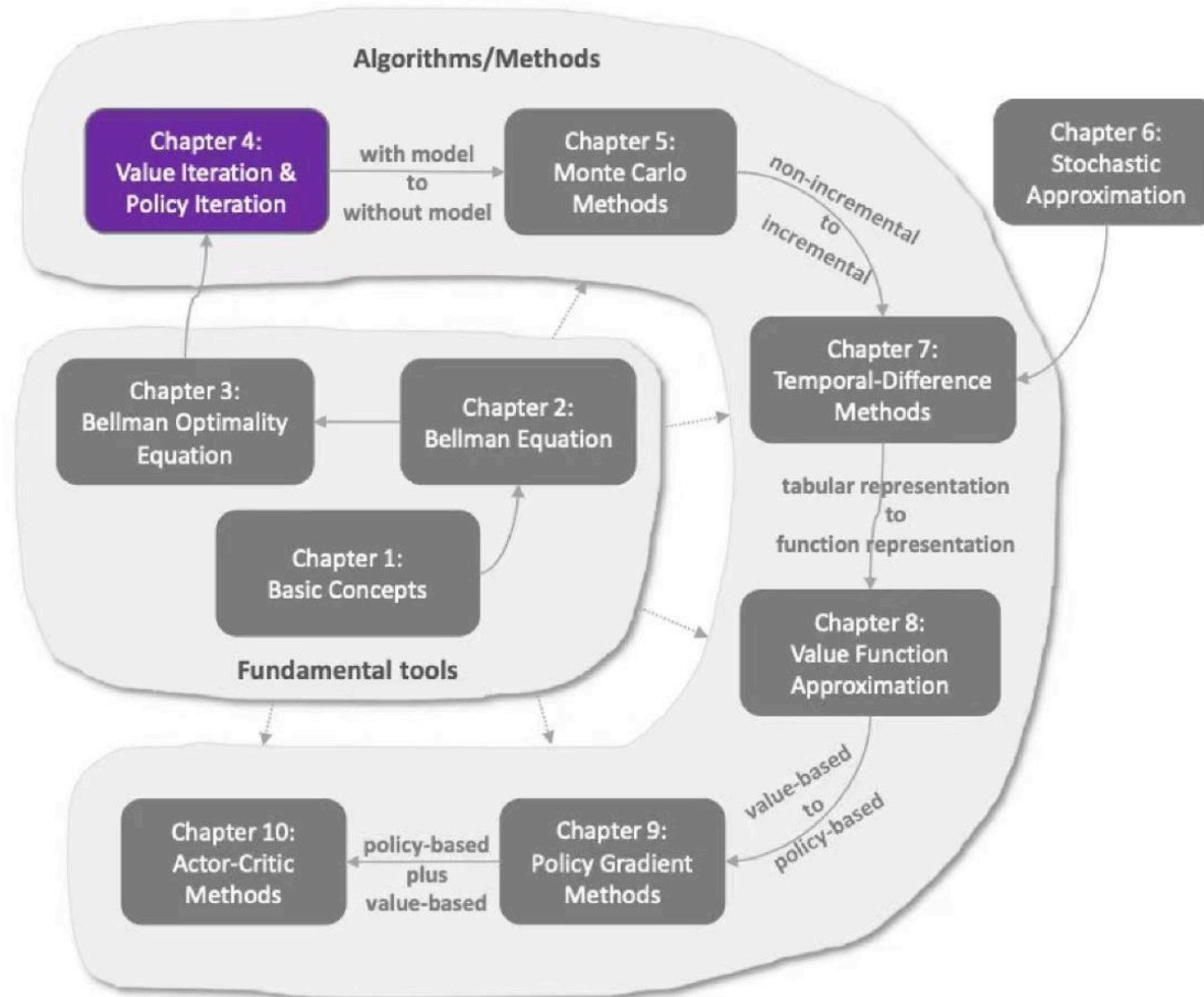
Most slides are adapted from UC Berkeley CS188/CS285, CMU 10-403

Recap: Markov Decision Process (MDP)

- Definition: (S, A, T, R, γ)
 - A set of states $s \in S$
 - A set of actions $a \in A$
 - A transition function $T(s, a, s')$
 - Probability that a from s leads to s' , i.e., $P(s'|s, a)$
 - Also called the model or the dynamics
 - A reward function $R(s, a, s')$
 - A discount factor γ
- Quantities:
 - Policy: map of states to actions
 - Utility: sum of discounted rewards
 - State values: expected future utility from a state
 - State-action (Q-)values: expected future utility from a state-action pair



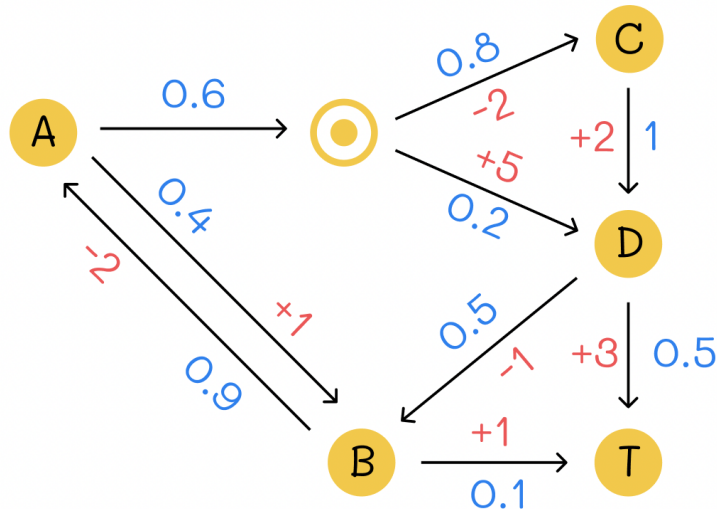
Dynamic



Solving Bellman Equations

• Example:

- Problem: Solve state value functions for each state
- Principle: There is a set of $|S|$ linear equations, one for each state, where V^π is its unique solution.



5 states (T is the terminal state)

- transition probability
- reward

Solve a system of equations consisting of $|S| = 5$ Bellman equations:

$$\begin{cases} V_A = 0.4 \cdot 1 \cdot (1 + V_B) + 0.6 \cdot [0.8 \cdot (-2 + V_C) + 0.2 \cdot (5 + V_D)] \\ V_B = 0.9 \cdot 1 \cdot (-2 + V_A) + 0.1 \cdot 1 \cdot (1 + V_T) \\ V_C = 1 \cdot 1 \cdot (2 + V_D) \\ V_D = 0.5 \cdot 1 \cdot (-1 + V_B) + 0.5 \cdot 1 \cdot (3 + V_T) \\ V_T = 0 \end{cases}$$

Since T is a terminal state, $V_T = 0$, so technically we only have to solve 4 equations

$$\begin{cases} V_A = 0.4V_B + 0.48V_C + 0.12V_D + 0.04 \\ V_B = 0.9V_A - 1.7 \\ V_C = V_D + 2 \\ V_D = 0.5V_B + 1 \\ V_T = 0 \end{cases} \Rightarrow \begin{cases} V_A = 1.11 \\ V_B = -0.7 \\ V_C = 2.65 \\ V_D = 0.65 \\ V_T = 0 \end{cases}$$

[Note] Analogous for solving the Q-function, but will be harder since we need to solve an equation for every state-action pair

Value Iteration

$$V^*(s) = \max_a \left(\sum_{s', r} p(s', r | s, a) [r + \gamma V^*(s')] \right)$$

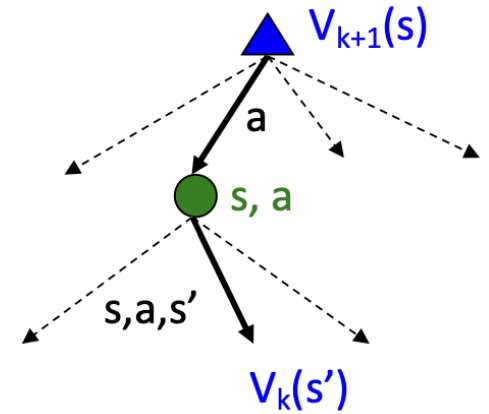
- It is recursive!
- The value of the current state $V^*(s)$ on the left side depends on the optimal value of the next state $V^*(s')$ on the right side.
 - We cannot know the current value without knowing the future value: a chicken-and-egg problem
- How do we actually compute this in practice? → Value Iteration

Value Iteration

- Define $V_k(s)$ to be the optimal value of s if the game ends in k more timesteps
- Start with $V_0(s) = 0$: zero utility if the game ends now
- Given vector of $V_k(s)$ values, we can compute $V_{k+1}(s)$

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

- Repeat until convergence, which yields V^*

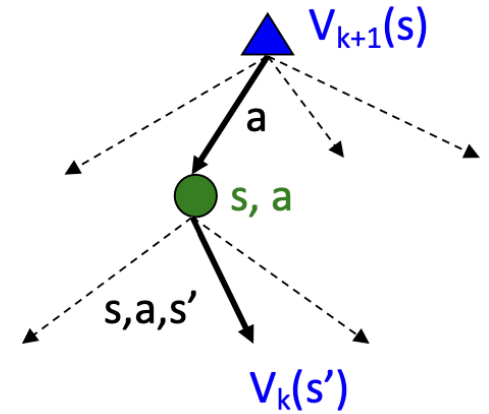


Value Iteration

- Define $V_k(s)$ to be the optimal value of s if the game ends in k more timesteps
- Start with $V_0(s) = 0$: zero utility if the game ends now
- Given vector of $V_k(s)$ values, we can compute $V_{k+1}(s)$

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

- Repeat until convergence, which yields V^*
- Question: what is the complexity of each iteration in terms of state space cardinality $|S|$ and action space cardinality $|A|$?



Value Iteration

Initialize $V(s)$ arbitrarily, for all $s \in \mathcal{S}$

Initialize θ to a small positive value

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \max_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

Until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi^*$, such that

$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$

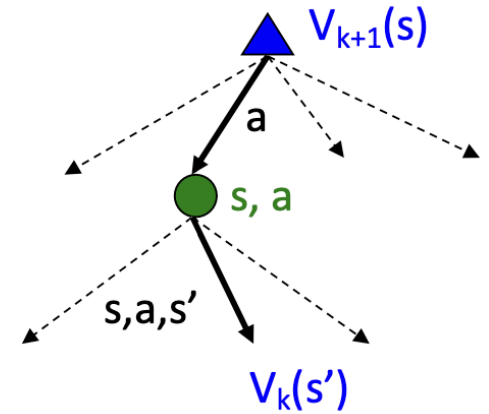
$\rightarrow O(|\mathcal{S}|^2 |A|)$ – as you need to operate on every state

Value Iteration

- Define $V_k(s)$ to be the optimal value of s if the game ends in k more timesteps
- Start with $V_0(s) = 0$: zero utility if the game ends now
- Given vector of $V_k(s)$ values, we can compute $V_{k+1}(s)$

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

- Repeat until convergence, which yields V^*
- Question: what is the complexity of each iteration in terms of state space cardinality $|S|$ and action space cardinality $|A|$?
 - $O(|S|^2|A|)$ – as you need to operate on every state

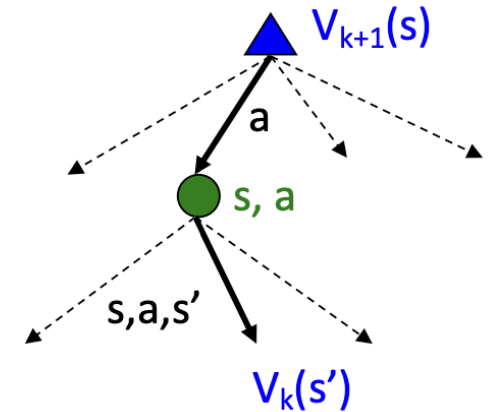


Value Iteration

- Define $V_k(s)$ to be the optimal value of s if the game ends in k more timesteps
- Start with $V_0(s) = 0$: zero utility if the game ends now
- Given vector of $V_k(s)$ values, we can compute $V_{k+1}(s)$

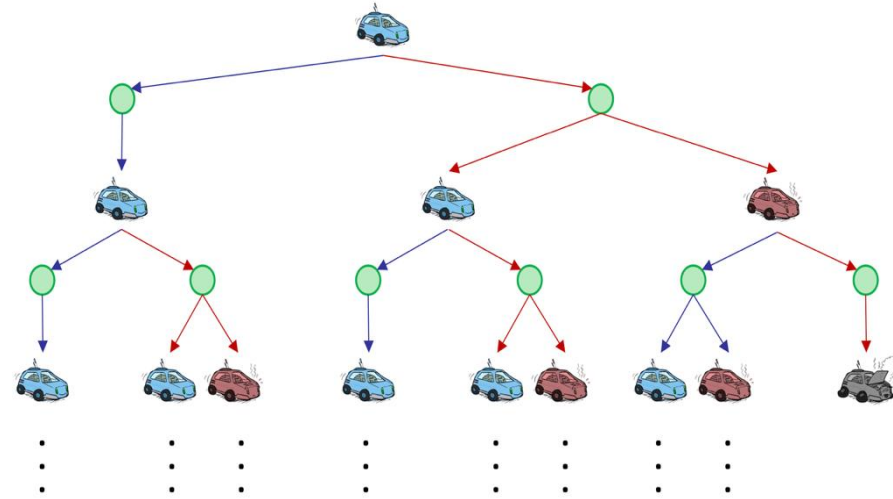
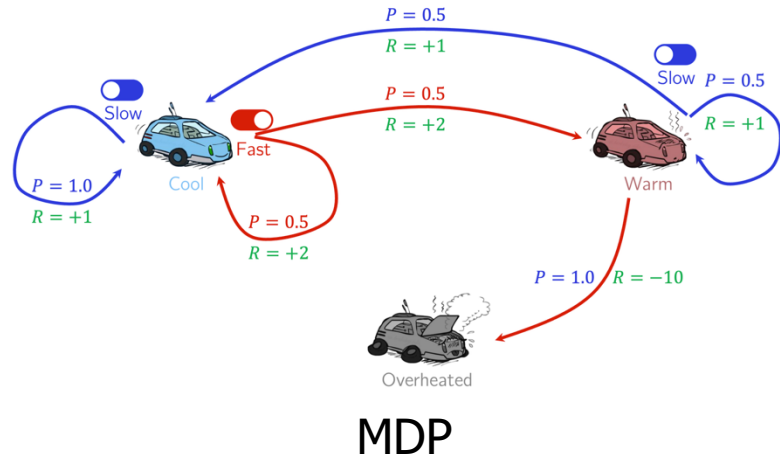
$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

- Repeat until convergence, which yields V^*
- Question: what is the complexity of each iteration in terms of state space cardinality $|S|$ and action space cardinality $|A|$?
 - $O(|S|^2|A|)$ – as you need to operate on every state
- Theorem: will converge to unique optimal values
 - Basic idea: approximations get refined towards optimal values
 - Policy may converge long before values do



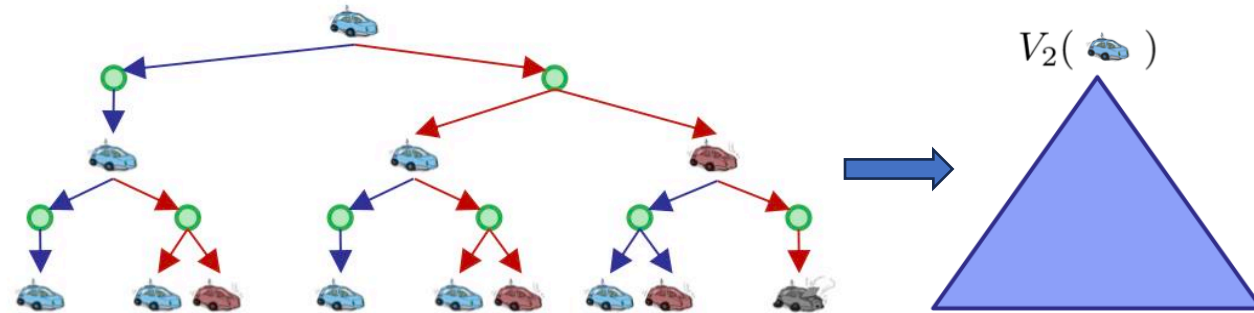
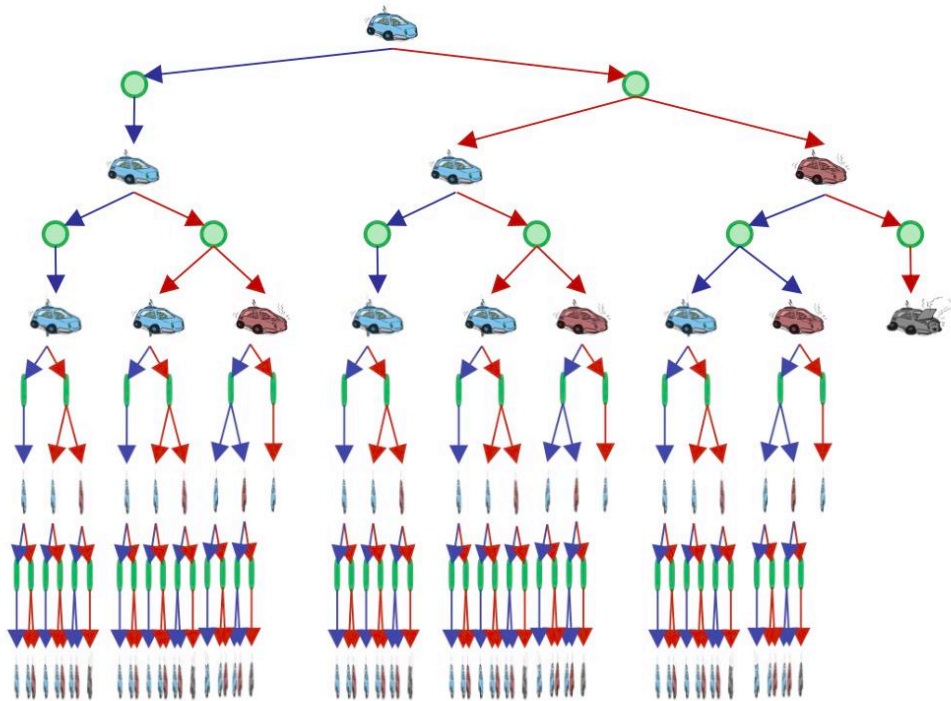
Value Iteration

- Example: Car racing



Value Iteration

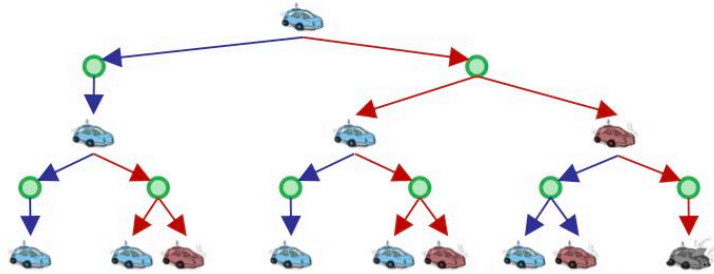
- Example: Car racing
 - Problem: tree goes on forever



- Idea: time-limited values
 - Define $V_k(s)$ to be the optimal value of s if the game ends in k more time steps

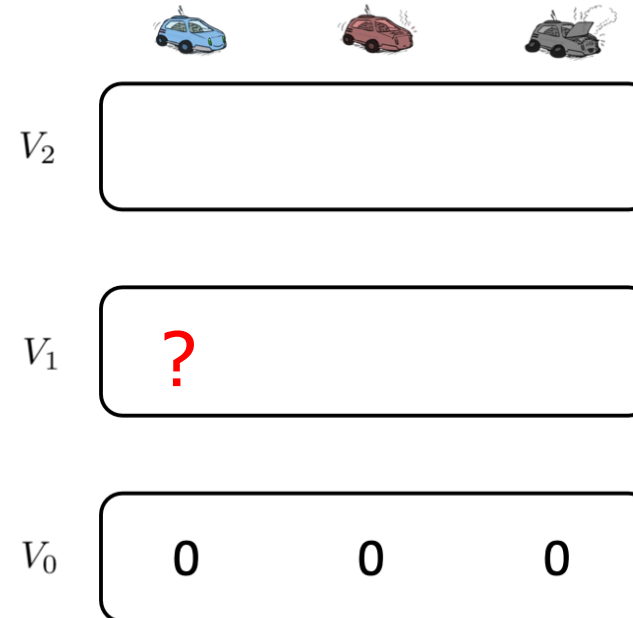
Value Iteration

- Example: Car racing

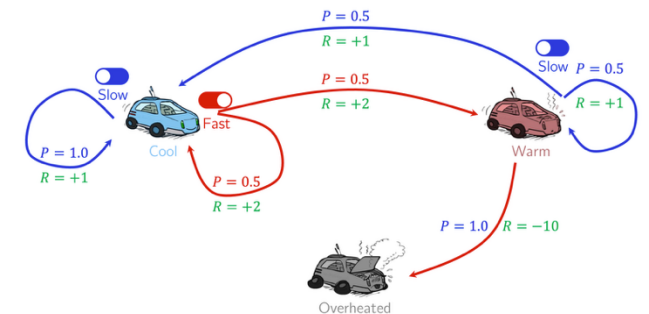


s	a	s'	$P(s' s, a)$	$R(s, a, s')$
Cool	Slow	Cool	1.0	+1
Cool	Fast	Cool	0.5	+2
Cool	Fast	Warm	0.5	+2
Warm	Slow	Cool	0.5	+1
Warm	Slow	Warm	0.5	+1
Warm	Fast	Overheated	1.0	-10
Overheated	(end)	Overheated	1.0	0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')] \quad (\text{assume no discount})$$

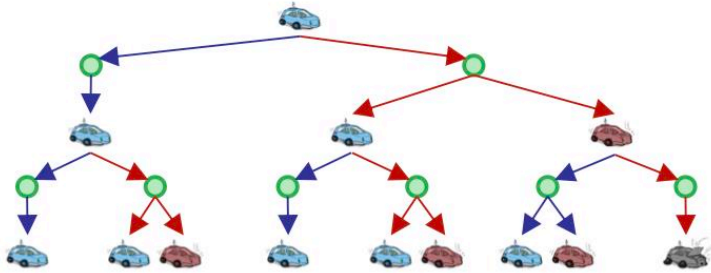


- action=slow:
 $1 \times (1 + 0) = 1$
- action=fast:
 $0.5 \times (2 + 0) + 0.5 \times (2 + 0) = 2$



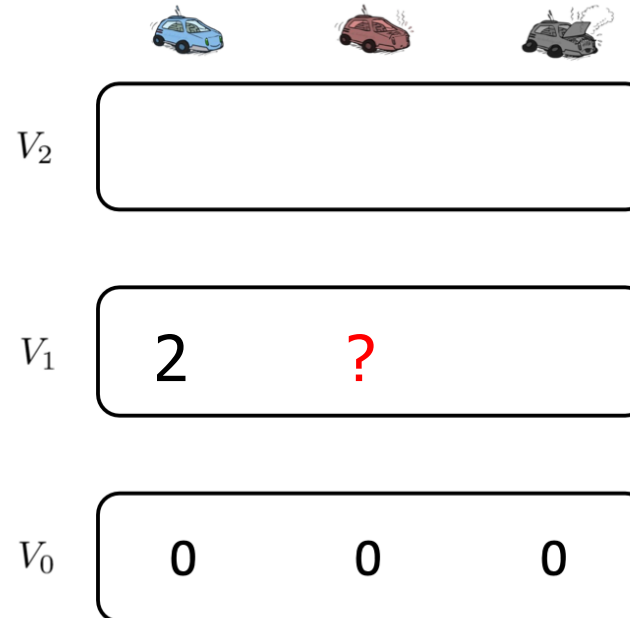
Value Iteration

- Example: Car racing

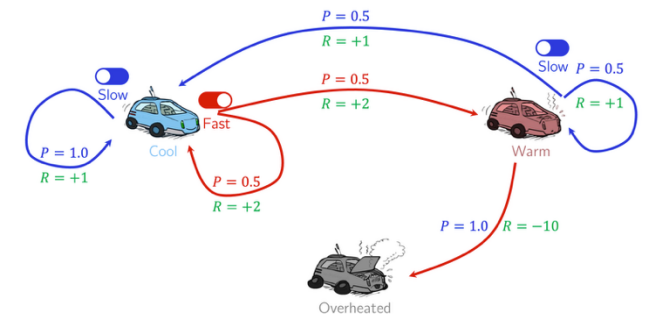


s	a	s'	$P(s' s, a)$	$R(s, a, s')$
 Cool	 Slow	 Cool	1.0	+1
 Cool	 Fast	 Cool	0.5	+2
 Cool	 Fast	 Warm	0.5	+2
 Warm	 Slow	 Cool	0.5	+1
 Warm	 Slow	 Warm	0.5	+1
 Warm	 Fast	 Overheated	1.0	-10
 Overheated	(end)	 Overheated	1.0	0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')] \quad (\text{assume no discount})$$

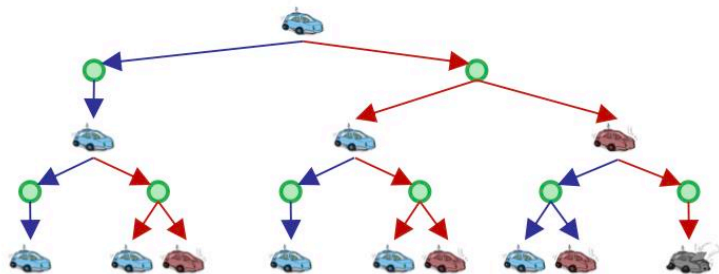


- **action=slow:**
 $0.5 \times (1 + 0) + 0.5 \times (1 + 0) = 1$
- **action=fast:**
 $1.0 \times (-10 + 0) = -10$



Value Iteration

- Example: Car racing



<i>s</i>	<i>a</i>	<i>s'</i>	<i>P(s' s, a)</i>	<i>R(s, a, s')</i>
Cool	Slow	Cool	1.0	+1
Cool	Fast	Cool	0.5	+2
Cool	Fast	Warm	0.5	+2
Warm	Slow	Cool	0.5	+1
Warm	Slow	Warm	0.5	+1
Warm	Fast	Overheated	1.0	-10
Overheated	(end)	Overheated	1.0	0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')] \quad (\text{assume no discount})$$

V_2

?

V_1

2

1

0

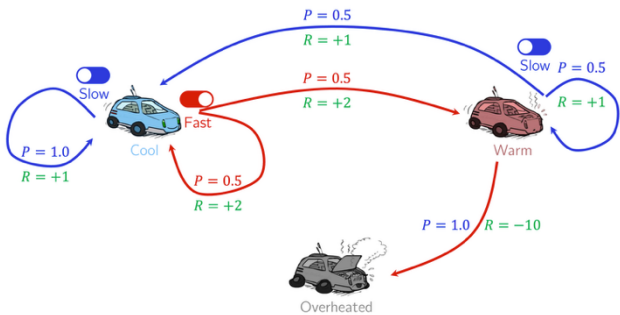
V_0

0

0

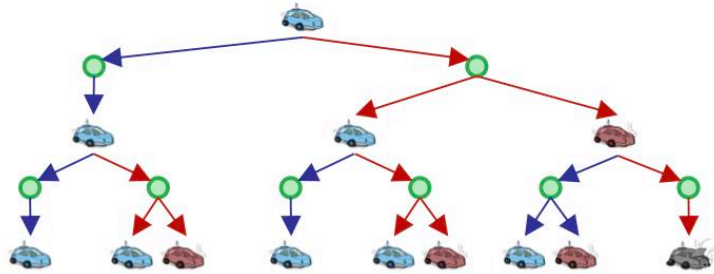
0

- action=slow:
 $1.0 \times (1 + 2) = 3$
- action=fast:
 $0.5 \times (2 + 2) + 0.5 \times (2 + 1) = 3.5$



Value Iteration

- Example: Car racing



s	a	s'	$P(s' s, a)$	$R(s, a, s')$
Cool	Slow	Cool	1.0	+1
Cool	Fast	Cool	0.5	+2
Cool	Fast	Warm	0.5	+2
Warm	Slow	Cool	0.5	+1
Warm	Slow	Warm	0.5	+1
Warm	Fast	Overheated	1.0	-10
Overheated	(end)	Overheated	1.0	0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')] \quad (\text{assume no discount})$$

V_2

3.5	?	

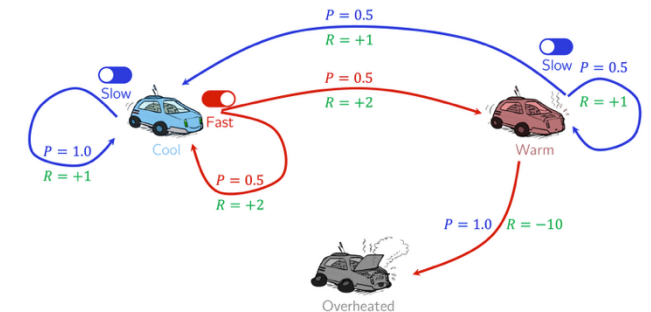
V_1

2	1	0

V_0

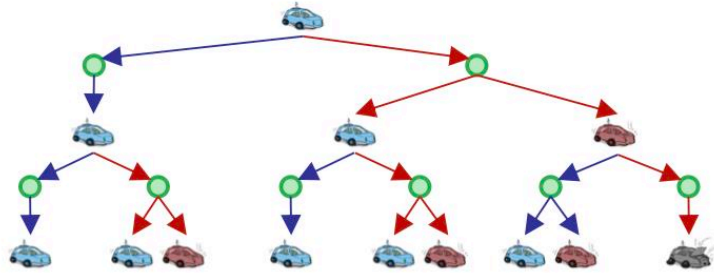
0	0	0

- action=slow:
 $0.5 \times (1 + 2) + 0.5 \times (1 + 1) = 2.5$
- action=fast:
 $1.0 \times (-10 + 0) = -10$



Value Iteration

- Example: Car racing



s	a	s'	$P(s' s, a)$	$R(s, a, s')$
Cool	Slow	Cool	1.0	+1
Cool	Fast	Cool	0.5	+2
Cool	Fast	Warm	0.5	+2
Warm	Slow	Cool	0.5	+1
Warm	Slow	Warm	0.5	+1
Warm	Fast	Overheated	1.0	-10
Overheated	(end)	Overheated	1.0	0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')] \quad (\text{assume no discount})$$

$$V_2$$

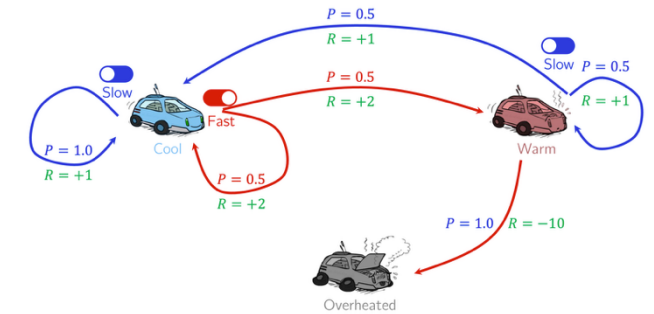
3.5	2.5	0

$$V_1$$

2	1	0

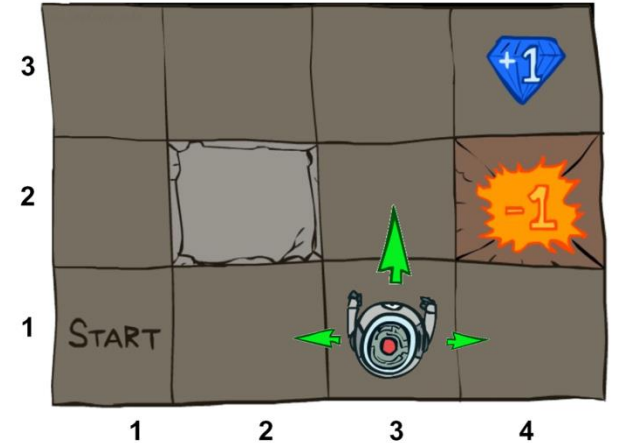
$$V_0$$

0	0	0



Value Iteration

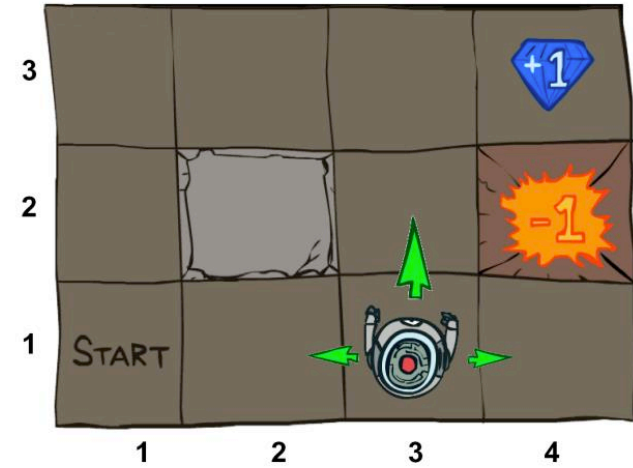
- Example: GridWorld
 - A maze-like problem
 - The agent lives in a grid
 - Walls block the agent's path
 - Noisy movement: actions do not always go as planned
 - 80% of the time, the action North takes the agent North (if there is no wall there)
 - 10% of the time, North takes the agent West;
 - 10% East
 - If there is a wall in the direction the agent would have been taken, the agent stays put
 - The agent receives rewards each time step
 - Small "living" reward each step (can be negative)
 - Big rewards come at the end (good or bad)
 - Goal: maximize sum of rewards



Value Iteration



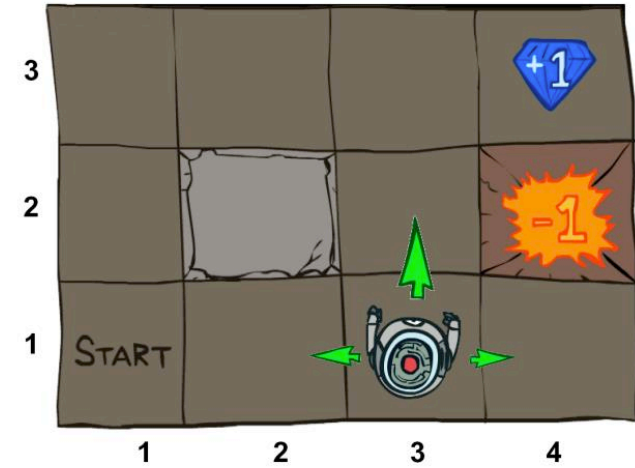
Noise = 0.2 Discount = 0.9 Living reward = 0
Initialization



Value Iteration



Noise = 0.2 Discount = 0.9 Living reward = 0
After 1 iteration



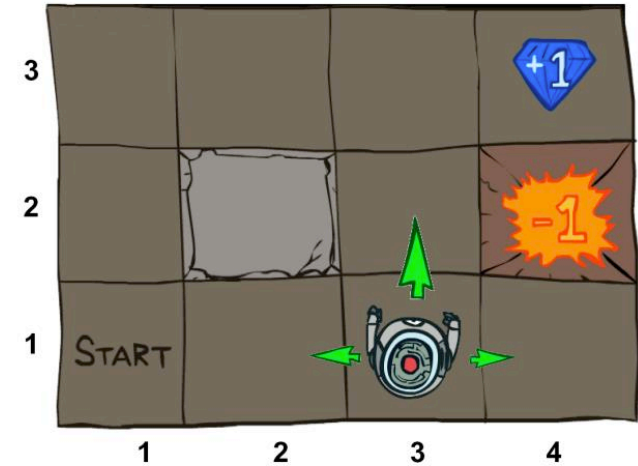
Value Iteration



Noise = 0.2 Discount = 0.9 Living reward = 0
After 2 iterations

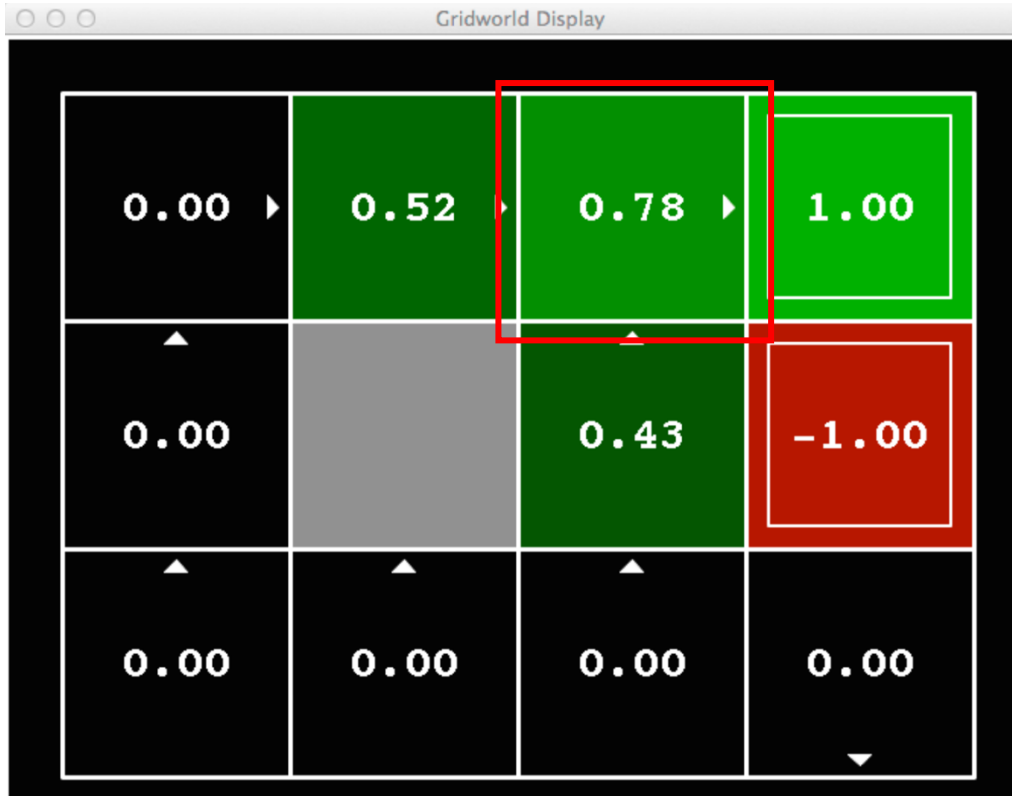
$$V_{k+1}(s = (3,3)) \leftarrow \max_a \sum_{s'} T(s = (3,3), a = \rightarrow, s') [R(s = (3,3), a = \rightarrow, s') + \gamma V_k(s')]$$

- action= $\rightarrow$: $(0.8 \times (0 + 0.9 \times 1) + 0.1 \times (0 + 0.9 \times 0) + 0.1 \times (0 + 0.9 \times 0)) = 0.72$

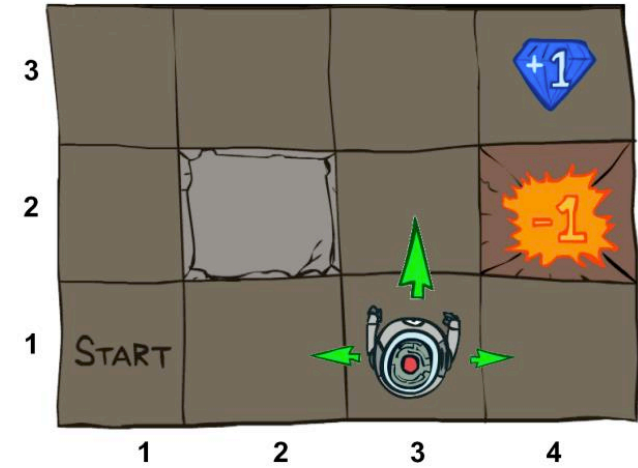


$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

Value Iteration

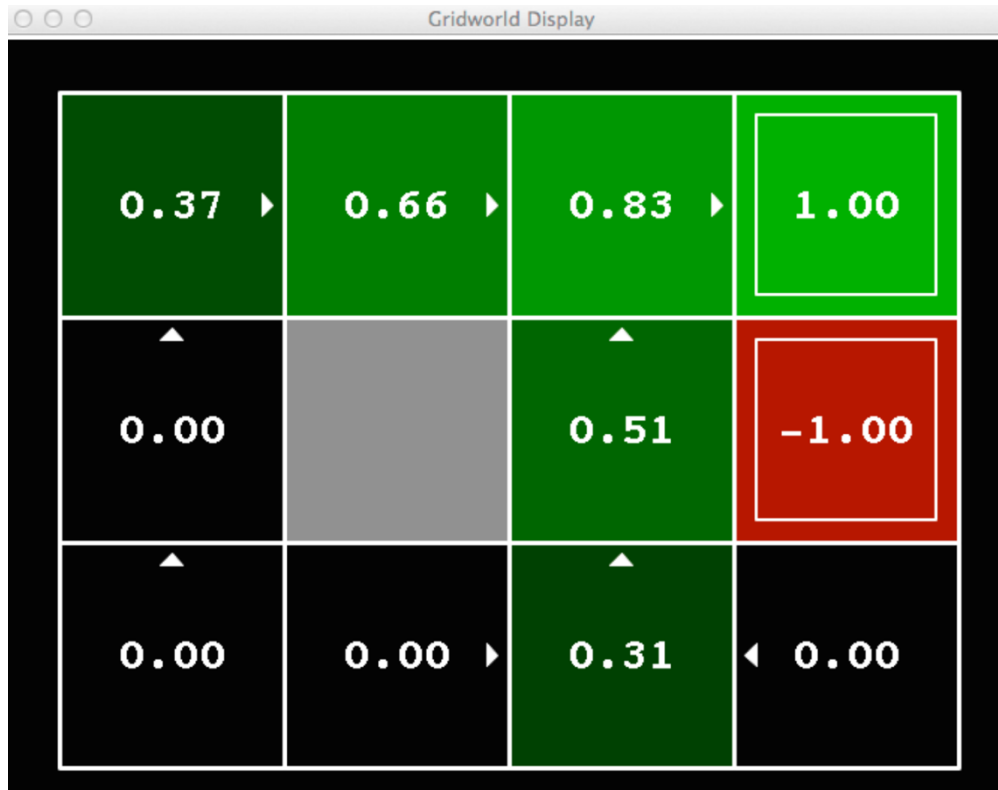


Noise = 0.2 Discount = 0.9 Living reward = 0
After 3 iterations

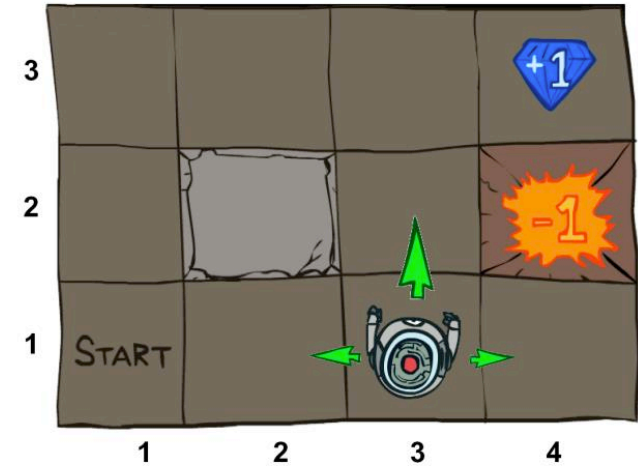


$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

Value Iteration



Noise = 0.2 Discount = 0.9 Living reward = 0
After 4 iterations



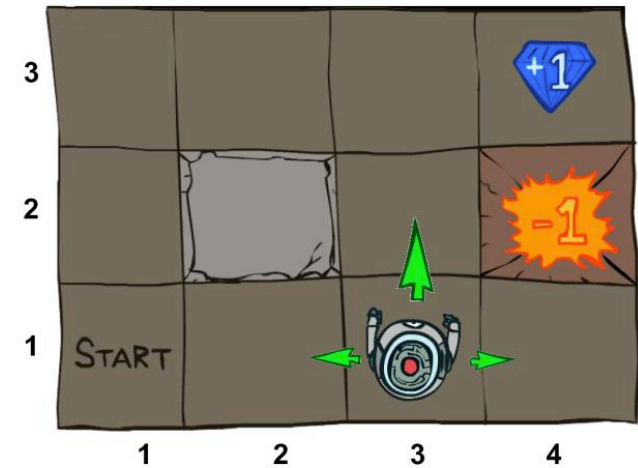
$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

Value Iteration



Noise = 0.2 Discount = 0.9 Living reward = 0

... At convergence



$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

- Information propagates outward from terminal states
- Eventually all states have correct value estimates

Value Iteration

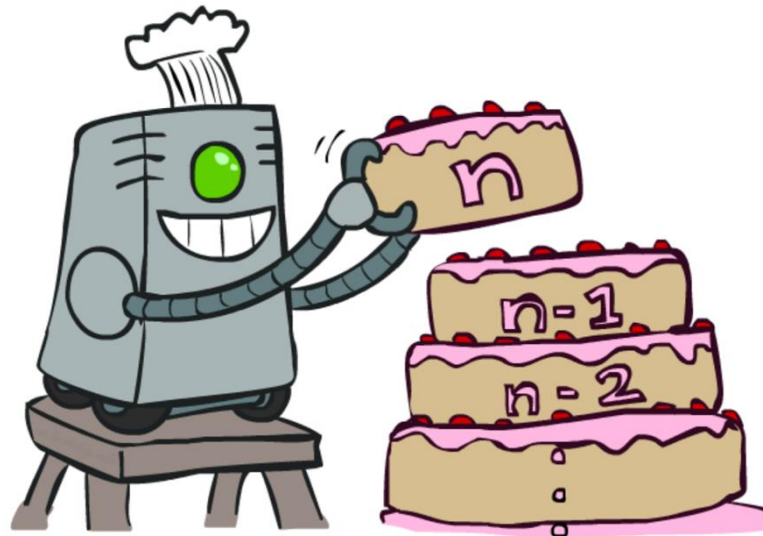
- Bellman equation **characterizes** the optimal values

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

- Value iteration **computes** the optimal values

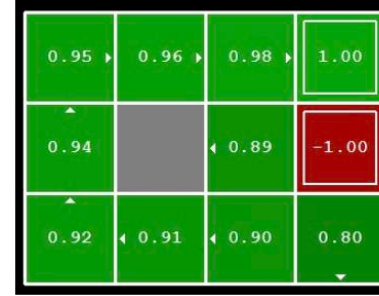
$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

- A fixed point solution method



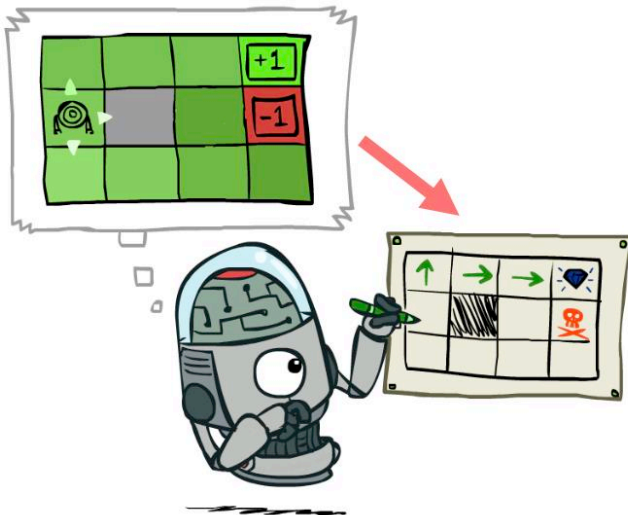
Computing Actions from Values

- Problem: how do we get actions?
 - Imagine we have the optimal values $V^*(s)$
 - How should we act?



- Policy extraction: gets the policy implied by the values
 - We need to do a one-step lookup

$$\pi^*(s) = \operatorname{argmax}_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$



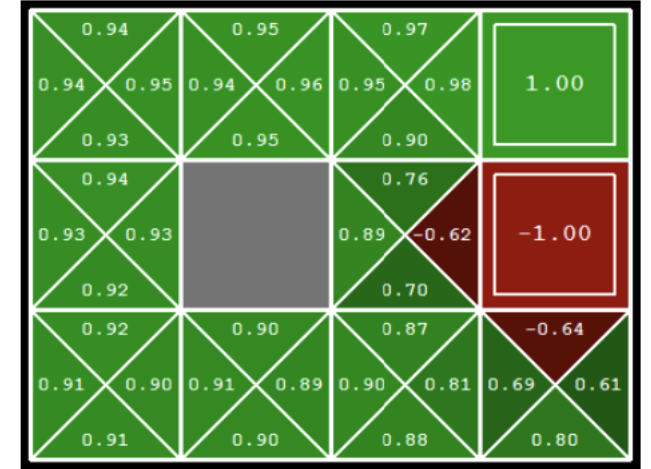
Computing Actions from Q-Values

- Imagine we have the optimal Q-values $Q^*(s)$
- How should we act?

- Trivial to decide by

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$$

→ Actions are easier to select from Q-values than state values

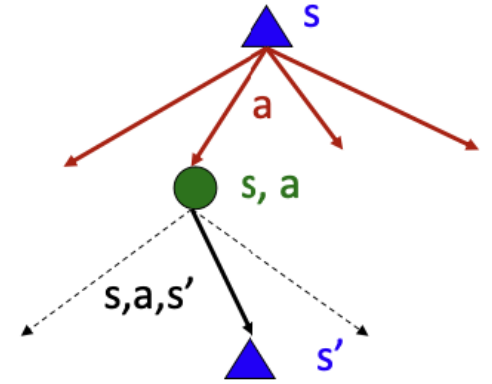


Problems with Value Iteration

- Value iteration repeats the Bellman update:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

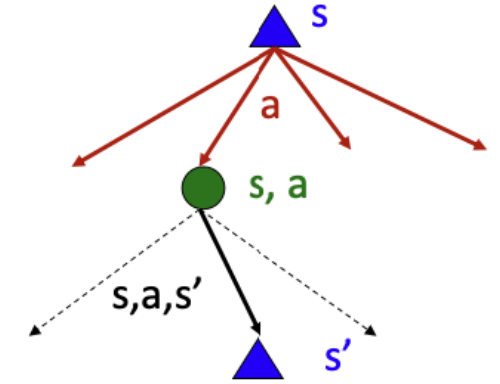
- Problem 1: slow – $O(|S|^2|A|)$ per iteration
- Problem 2: The policy often converges long before the values
 - [Demo](#)



Problems with Value Iteration

- Value iteration repeats the Bellman update:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$



- Problem 1: slow – $O(|S|^2|A|)$ per iteration
- Problem 2: The policy often converges long before the values



→ Policy Methods

Thanks!